SQL ASSIGNMENT

Questions

Scenario: A company wants to analyse employee assignments across departments and projects. It also needs a reusable stored procedure that adds a surrogate key to a selected table and records session-level and execution-level logs for every run.

Question 1 - Create the Database Tables

Create the following four related business tables in the dbo schema:

Table	Required Columns	Relationship
Departments	DepartmentId, DepartmentName	DepartmentId is the primary key.
Employees	EmployeeId, EmployeeName, DepartmentId, Salary	DepartmentId references Departments.
Projects	ProjectId, ProjectName, DepartmentId	DepartmentId references Departments.
EmployeeProjects	EmployeeId, ProjectId, HoursWorked	EmployeeId and ProjectId reference their parent tables.

- Insert at least 3 departments, 6 employees, 4 projects, and 8 employee-project assignments.
- Include at least one project with more than 100 total working hours and at least two assigned employees.

Question 2 - Create a Multi-Table CTE Report

Create a CTE named DepartmentProjectSummary by joining Departments, Employees, Projects, and EmployeeProjects.

- Return DepartmentName and ProjectName.
- Calculate the distinct number of assigned employees as TotalEmployees.
- Calculate the sum of HoursWorked as TotalHours.
- Calculate the average employee salary as AverageSalary and round it to two decimal places.
- Return only projects where TotalHours is greater than 100 and TotalEmployees is at least 2.
- Sort the final result by TotalHours in descending order.

SECTION 1

Question 3 - Create the Logging Tables

Create the following two tables to record every execution of the surrogate-key procedure:

Table	Required Fields	Purpose
SessionLog	SessionLogId, SessionId, UserName, SessionStartTime, SessionEndTime, SessionStatus, FinalMessage	Stores one record for the complete procedure call.
ExecutionLog	ExecutionLogId, SessionLogId, ProcedureName, TableName, OperationName, ExecutionStartTime, ExecutionEndTime, ExecutionStatus, RowsAffected, ErrorNumber, ErrorMessage	Stores the individual operation performed during the session.

- Use identity columns as the primary keys of both log tables.
- Create a foreign key from ExecutionLog.SessionLogId to SessionLog.SessionLogId.
- Use Running, Completed, Skipped, and Failed as valid status values.

Question 4 - Create a Generic Surrogate-Key Stored Procedure

Create dbo.usp_AddSurrogateKey. The caller must provide only the table name:

```
EXEC dbo.usp_AddSurrogateKey @TableName = 'EmployeeProjects';
```

- Accept @TableName as SYSNAME.
- Check that dbo.<TableName> exists.
- Check whether a column named SurrogateKey already exists.
- Add SurrogateKey BIGINT IDENTITY(1,1) NOT NULL only when it is missing.
- Do not remove, replace, or modify an existing primary key.
- When the table has no primary key, create a primary key on SurrogateKey. Otherwise, create a unique index or unique constraint on SurrogateKey.
- Use safe dynamic SQL with metadata validation and QUOTENAME.
- Use a transaction and TRY...CATCH error handling.
- Return a clear success, skipped, or failure message.

SECTION 1

Question 5 - Assign Session and Execution Logs

Integrate both logs inside dbo.usp_AddSurrogateKey using the following sequence:

Step	Stage	Required Log Action
1	At procedure start	Insert one SessionLog record with Running status and capture SessionLogId using SCOPE_IDENTITY().
2	Before validation/operation	Insert one related ExecutionLog record with Running status.
3	On successful completion	Update ExecutionLog and SessionLog to Completed and set their end times.
4	When the column already exists	Update both records to Skipped and store a meaningful message.
5	On unexpected error	Roll back the active transaction, update both records to Failed, save ERROR_NUMBER() and ERROR_MESSAGE(), and re-throw the error.

- Use SUSER_SNAME() or ORIGINAL_LOGIN() to record the executing user.
- Use either SYSUTCDATETIME() or SYSDATETIME() consistently for every timestamp.
- RowsAffected should represent the number of existing rows that received generated identity values.

Question 6 - Test and Verify the Solution

- Execute the procedure for EmployeeProjects and confirm that SurrogateKey is added.
- Execute the procedure again for EmployeeProjects and confirm that no duplicate column is created.
- Execute the procedure for UnknownTable and confirm that the failure or skipped result is logged clearly.
- Query SessionLog and ExecutionLog to verify the relationship, status, timestamps, messages, and error details.
- Display the CTE output and the updated EmployeeProjects table.

SECTION 2

Deliverables

Submit the following completed items. All deliverables must be based on the same database and must run in Microsoft SQL Server.

Deliverable	Required Content
1. SQL Script	One complete .sql file containing table creation, constraints, sample data, the CTE query, both logging tables, dbo.usp_AddSurrogateKey, test executions, and verification queries.
2. CTE Result	Evidence showing DepartmentName, ProjectName, TotalEmployees, TotalHours, and AverageSalary with the required filtering and sorting.
3. Surrogate-Key Tests	Evidence of a successful execution, a repeated execution where the key already exists, and an invalid-table execution.
4. Log Results	Evidence from SessionLog and ExecutionLog showing linked records, correct statuses, timestamps, messages, rows affected, and error details where applicable.
5. Script Quality	Clear comments before each question, consistent object names, safe dynamic SQL, rerun-friendly organisation, and no hard-coded object IDs.

Completion Checklist

- ☐ All four business tables are created with valid relationships.
- ☐ The CTE uses all four business tables and returns the required calculations.
- ☐ SurrogateKey is added only once and existing primary keys remain unchanged.
- ☐ The table name is validated and QUOTENAME is used in dynamic SQL.
- ☐ Every stored-procedure call creates one SessionLog record and one related ExecutionLog record.
- ☐ TRY...CATCH and transaction handling leave the database in a consistent state.
- ☐ All three required procedure tests are demonstrated.

Important: The assignment requires the solution script and execution evidence. Do not submit only screenshots.

SECTION 3

Submission

Required File Name

Use the following naming format:

```
StudentName_SQL_Practical_Assignment.sql
```

Submission Package

- Submit the .sql file as the main file.
- Attach result screenshots or exported result grids in one PDF only when requested by the reviewer.
- The submitted script must run in the order presented, from table creation through verification.
- Remove temporary test objects that are not part of the assignment.
- Do not include passwords, connection strings, or confidential environment details.